System Design: Amazon Shopping Cart

E-commerce Cart Service — Senior Engineer Interview Walkthrough

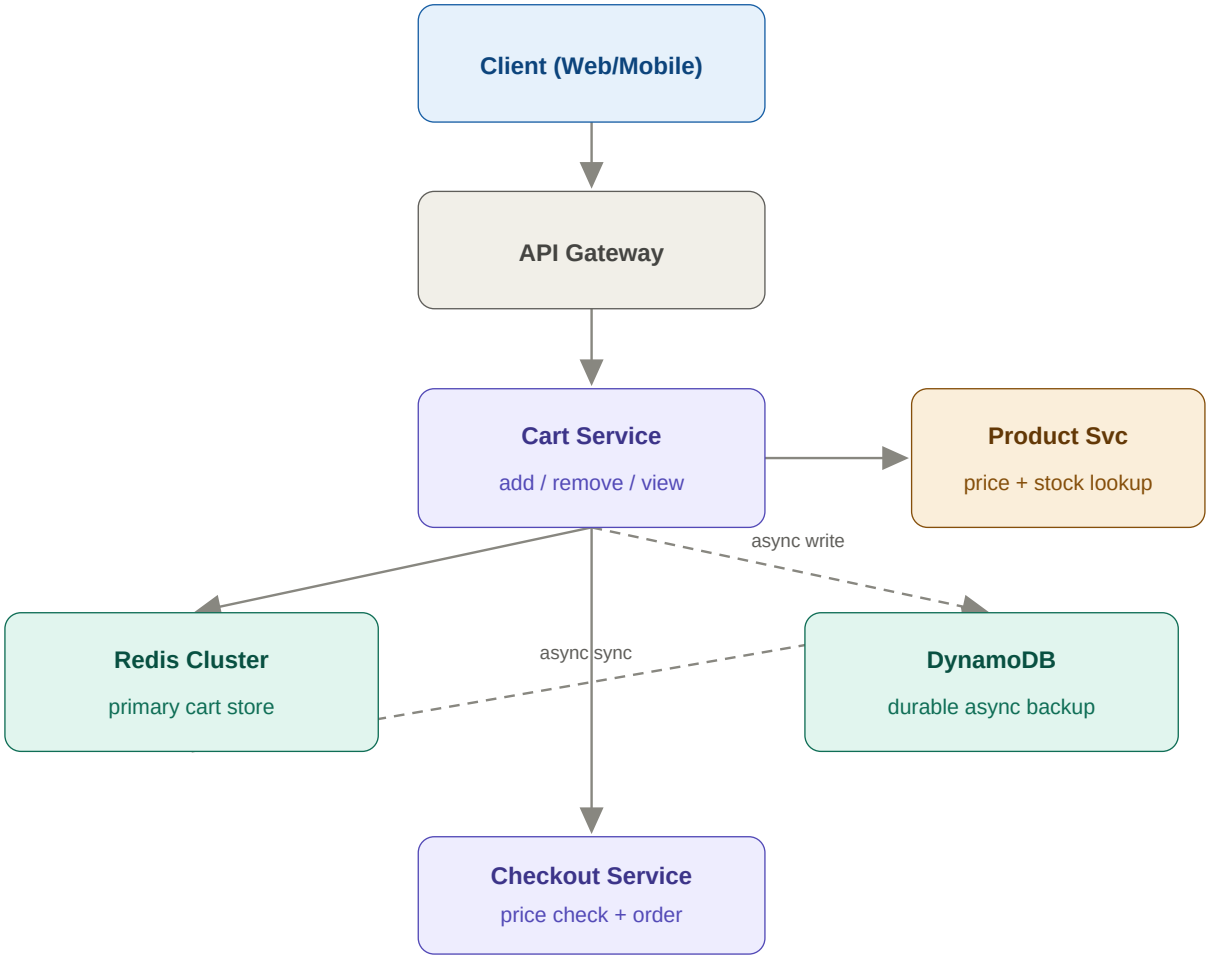
1. Requirements & Capacity

Think out loud: "Does cart persist on logout? Guest cart? Does adding to cart reserve inventory? Price changes while in cart? I'll assume: 30-day persist for logged-in, guest in browser, NO inventory reservation, price fetched fresh at checkout."

Scale	Users/RPS	Architecture	Storage
Low	1M users 100 RPS	Postgres cart table + app server	~10 GB
Mid	50M users 5k RPS	Redis cart cache + Postgres backup + cart microservice	~500 GB
High	300M users 170k RPS peak (Prime Day)	Redis cluster + async DynamoDB backup + CDN	~3 TB Redis

2. Architecture Diagram

Think out loud: "Cart primary store = Redis Hash (user_id → {product_id: {qty, added_at}}). DynamoDB = durable async backup. Product Service = price enrichment at display time. Cart itself stores NO prices."



hitecture: Redis is the primary cart store (<1ms reads). DynamoDB is the durable backup written asynchronously. Product Service enriches cart with current prices on every re

3. DB Schema

carts (DynamoDB)		
PK	**user_id**	*STRING*
	items	*MAP<prod_id,CartItem>*
	updated_at	*TIMESTAMP*
	ttl	*NUMBER (auto-expire)*

cart_items (Redis Hash)	
key: cart:{user_id}	*Redis Hash*
field: product_id	*String*
value: {qty, added_at}	*JSON string*
TTL	*30 days (auto-reset)*

orders (at checkout)		
PK	**order_id**	*UUID*
FK	user_id	*BIGINT*
	items_snapshot	*JSONB*
	total_price	*DECIMAL*
	status	*ENUM*
	created_at	*TIMESTAMP*

Schema *out time.*

Why this choice: Redis Hash HSET cart:{user_id} product_id '{"qty":2,...}'. Add item = HSET O(1). Remove = HDEL O(1). View cart = HGETALL one round-trip. TTL auto-handles 30-day expiry.

Why no price in cart: Cart stores product_id + quantity ONLY — never price. Price is fetched fresh from Product Service on every cart display. This prevents users being charged wrong prices after promotions change.

4. Guest Cart + Prime Day Scaling

- Guest cart: browser localStorage as JSON. On login, POST /api/v1/cart/merge sends {items:[...]}. Server merges — existing items keep higher qty.
- Why client-side for guests: zero server storage cost. Most guests never convert.
- Prime Day (10x traffic): Redis cluster with 10 shards (hash-slot by user_id). Each shard handles ~17k RPS — well within Redis limits.
- Pre-warm: load top 10M carts from DynamoDB into Redis 2 hours before sale. Avoids cold cache miss storm at launch.

GET	`/api/v1/cart`	Returns enriched cart (items + current prices)
PUT	`/api/v1/cart/items/{product_id}`	{qty} — add or update
DELETE	`/api/v1/cart/items/{product_id}`	Remove single item
POST	`/api/v1/cart/merge`	{items:[{product_id,qty}]} — guest merge on login

DELETE	`/api/v1/cart`	Clear cart (post-checkout)

Decision	Choice	Gain	Trade-off
Primary store	Redis	Sub-ms, simple ops	Volatile, memory cost
Backup	DynamoDB	Durable, serverless	Async — tiny staleness
Price	Not in cart	Always current price	Extra lookup on display
Guest cart	Client-side	Zero server cost	Lost on device change